

Mip-Splatting: Alias-free 3D Gaussian Splatting

Zehao Yu^{1,2} Anpei Chen^{1,2} Binbin Huang³ Torsten Sattler⁴ Andreas Geiger^{1,2}

¹University of Tübingen ²Tübingen AI Center ³ShanghaiTech University
⁴Czech Technical University in Prague

<https://niu jinshuchong.github.io/mip-splatting>

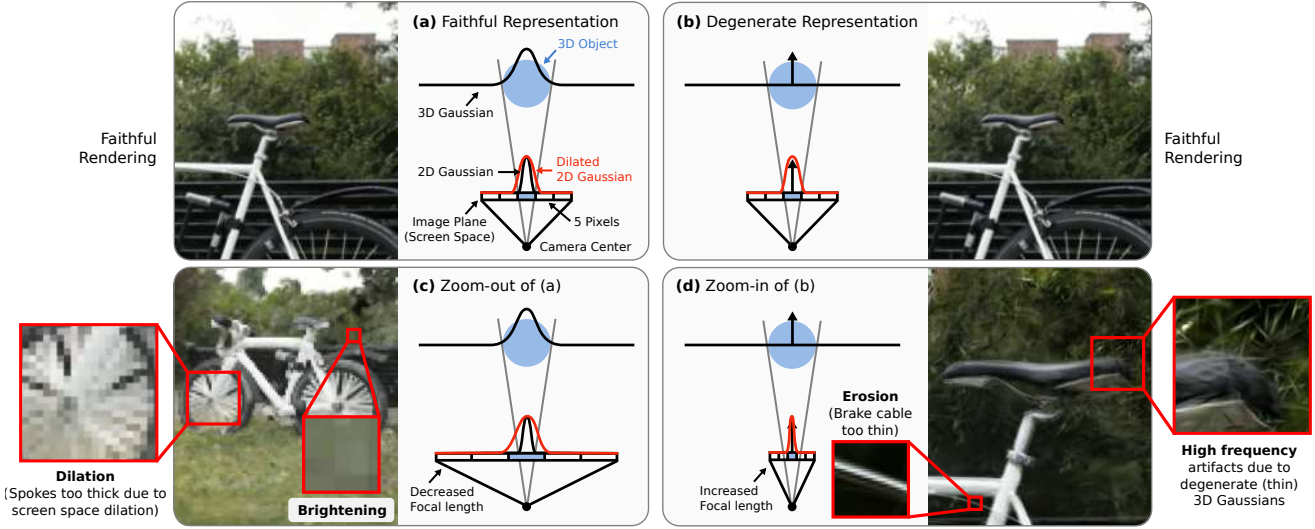


Figure 1. **3D Gaussian Splatting** [18] renders images by representing **3D Objects** as **3D Gaussians** which are projected onto the image plane followed by **2D Dilation** in screen space as shown in (a). The method’s intrinsic shrinkage bias leads to degenerate 3D Gaussians exceed sampling limit as illustrated by the δ function in (b) while rendering similarly to 2D due to the dilation operation. However, when changing the sampling rate (via the focal length or camera distance), we observe strong dilation effects (c) and high frequency artifacts (d).

Abstract

Recently, *3D Gaussian Splatting* has demonstrated impressive novel view synthesis results, reaching high fidelity and efficiency. However, strong artifacts can be observed when changing the sampling rate, e.g., by changing focal length or camera distance. We find that the source for this phenomenon can be attributed to the lack of 3D frequency constraints and the usage of a 2D dilation filter. To address this problem, we introduce a 3D smoothing filter which constrains the size of the 3D Gaussian primitives based on the maximal sampling frequency induced by the input views, eliminating high-frequency artifacts when zooming in. Moreover, replacing 2D dilation with a 2D Mip filter, which simulates a 2D box filter, effectively mitigates aliasing and dilation issues. Our evaluation, including scenarios such as a training on single-scale images and testing on multiple scales, validates the effectiveness of our approach.

1. Introduction

Novel View Synthesis (NVS) plays a critical role in computer graphics and computer vision, with various applications including virtual reality, cinematography, robotics, and more. A particularly significant advancement in this field is the Neural Radiance Field (NeRF) [28], introduced by Mildenhall et al. in 2020. NeRF utilizes a multi-layer perceptron (MLP) to represent geometry and view-dependent appearance effectively, demonstrating remarkable novel view rendering quality. Recently, 3D Gaussian Splatting (3DGS) [18] has gained attention as an appealing alternative to both MLP [28] and feature grid-based representations [4, 11, 24, 32, 46]. 3DGS stands out for its impressive novel view synthesis results, while achieving real-time rendering at high resolutions. This effectiveness and efficiency, coupled with the potential integration into the standard rasterization pipeline of GPUs represents a signif-

icant step towards practical usage of NVS methods.

Specifically, 3DGS represents complex scenes as a set of 3D Gaussians, which are rendered to screen space through splatting-based rasterization. The attributes of each 3D Gaussian, i.e., position, size, orientation, opacity, and color, are optimized through a multi-view photometric loss. Thereafter, a 2D dilation operation is applied in screen space for low-pass filtering. Although 3DGS has demonstrated impressive NVS results, it produces artifacts when camera views diverge from those seen during training, such as zoom in and zoom out, as illustrated in Figure 1. We find that the source for this phenomenon can be attributed to the lack of 3D frequency constraints and the usage of a 2D dilation filter. Specifically, zooming out leads to a reduced size of the projected 2D Gaussians in screen space, while applying the same amount of dilation results in dilation artifacts. Conversely, zooming in causes erosion artifacts since the projected 2D Gaussians expand, yet dilation remains constant, causing erosion and resulting in incorrect gaps between Gaussians in the 2D projection.

To resolve these issues, we propose to regularize the 3D representation in 3D space. Our key insight is that the highest frequency that can be reconstructed of a 3D scene is inherently constrained by the sampling rates of the input images. We first derive the multi-view frequency bounds of each Gaussian primitive based on the training views according to the Nyquist-Shannon Sampling Theorem [33, 45]. By applying a low-pass filter to the 3D Gaussian primitives in 3D space during the optimization, we effectively restrict the maximal frequency of the 3D representation to meet the Nyquist limit. Post-training, this filter becomes an intrinsic part of the scene representation, remaining constant regardless of viewpoint changes. Consequently, our method eliminates the artifacts presents in 3DGS [18] when zooming in, as shown in the $8\times$ higher resolution image in Figure 2.

Nonetheless, rendering the reconstructed scene at lower sampling rates (e.g., zooming out) results in aliasing. Previous work [1–3, 17] address aliasing by employing cone tracing and applying pre-filtering to the input positional or feature encoding, which is not applicable to 3DGS. Thus, we introduce a 2D Mip filter (à la “mipmap”) specifically designed to ensure alias-free reconstruction and rendering across different scales. Our 2D Mip filter mimics the 2D box filter inherent to the actual physical imaging process [29, 37, 48], by approximating it with a 2D Gaussian low pass filter. In contrast to previous work [1–3, 17] that rely on the MLP’s ability to interpolate multi-scale signals during training with multi-scale images, our closed-form modification to the 3D Gaussian representation results in excellent out-of-distribution generalization: Training at a single sampling rate enables faithful rendering at various sampling rates different from those used during training as demonstrated by the $1/4\times$ down-sampled image in Figure 2.

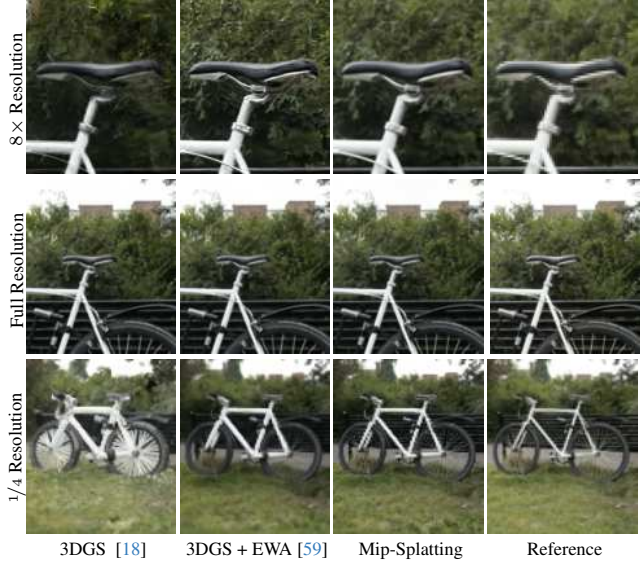


Figure 2. We trained all the models on single-scale (full resolution here) images and rendered images with different resolutions by changing focal length. While all methods show similar performance at training scale, we observe strong artifacts in previous work [18, 59] when changing the sampling rate. By contrast, our Mip-Splatting renders faithful images across different scales.

In summary, we make the following contributions:

- We introduce a **3D smoothing filter** for 3DGS to effectively regularize the maximum frequency of 3D Gaussian primitives, resolving the artifacts observed in out-of-distribution renderings of prior methods [18, 59].
- We replace the 2D dilation filter with a **2D Mip filter** to address aliasing and dilation artifacts.
- Experiments on challenging benchmark datasets [2, 28] demonstrate the effectiveness of Mip-Splatting when modifying the sampling rate.
- Our modifications to 3DGS are principled and simple, requiring only few changes to the original 3DGS code.

2. Related Work

Novel View Synthesis: NVS is the process of generating new images from viewpoints different from those of the original captures [12, 22]. NeRF [28], which leverages volume rendering [10, 21, 25, 26], has become a standard technique in the field. NeRF utilizes MLPs [5, 27, 34] to model scenes as continuous functions, which, despite their compact representation, impede rendering speed due to the expensive MLP evaluation that is required for each ray point. Subsequent methods [16, 40, 41, 52, 54] distill a pretrained NeRF into a sparse representation, enabling real-time rendering of NeRFs. Further advancements have been made to improve the training and rendering of NeRF with advanced scene representations [4, 6, 11, 18, 19, 24, 32, 46, 51]. In particular, 3D Gaussians Splatting (3DGS) [18]

demonstrated impressive novel view synthesis results, while achieving real-time rendering at high-definition resolutions. Importantly, 3DGS represents the scene explicitly as a collection of 3D Gaussians and uses rasterization instead of ray tracing. Nevertheless, 3DGS focuses on in-distribution evaluation where training and testing are conducted at similar sampling rates (focal length/scene distance). In this paper, we study the out-of-distribution generalization of 3DGS, training models at a single scale and evaluating it across multiple scales.

Primitive-based Differentiable Rendering: Primitive-based rendering techniques, which rasterize geometric primitives onto the image plane, have been explored extensively due to their efficiency [13, 14, 38, 44, 59, 60]. Differentiable point-based rendering methods [20, 36, 39, 43, 49, 53, 57] offer great flexibility in representing intricate structures and are thus well-suited for novel view synthesis. Notably, Pulsar [20] stands out for its efficient sphere rasterization. The more recent 3D Gaussian Splatting (3DGS) work [18] utilizes anisotropic Gaussians [59] and introduces a tile-based sorting for rendering, achieving remarkable frame rates. Despite its impressive results, 3DGS exhibits strong artifacts when rendering at a different sampling rate. We address this issue by introducing a 3D smoothing filter to constrain the maximal frequencies of the 3D Gaussian primitive representation, and a 2D Mip filter that approximates the box filter of the physical imaging process for alias-free rendering.

Anti-aliasing in Rendering: There are two principal strategies to combat aliasing: *super-sampling*, which increases the number of samples [7], and *prefiltering*, which applies low-pass filtering to the signal to meet the Nyquist limit [8, 15, 31, 47, 50, 59]. For example, EWA splatting [59] applies a Gaussian low pass filter to the projected 2D Gaussian in screen space to produce a band limited output respecting the Nyquist frequency of the image. While we also apply a band-limited filter to the Gaussian primitives, our band-limited filter is applied in 3D space and the filter size is fully determined by the training images not the images to be rendered. While our 2D Mip filter is also a Gaussian low pass filter in screen space, it approximates the box filter of the physical imaging process, approximating a single pixel. Conversely, the EWA filter limits the frequency signal’s bandwidth to the rendered image, and the size of the filter is chosen empirically. A critical difference to [59] is that we tackle the reconstruction problem, optimizing the 3D Gaussian representation via inverse rendering while EWA splatting only considers the rendering problem.

Recent neural rendering methods integrate pre-filtering to mitigate aliasing [1–3, 17, 58]. Mip-NeRF [1], for instance, introduced an integrated position encoding (IPE) to attenuate high-frequency details. A similar idea is adapted

for feature grid-based representations [3, 17, 58]. However, these approaches require multi-scale images for super-resolution. In contrast, our approach is based on 3DGS [18] and determines the necessary low-pass filter size based on pixel size, allowing for alias-free rendering at scales unobserved during training.

3. Preliminaries

In this section, we first review the sampling theorem in Section 3.1, laying the foundation for understanding the aliasing problem. Subsequently, we introduce 3D Gaussian Splatting (3DGS) [18] and its rendering process in Section 3.2.

3.1. Sampling Theorem

The Sampling Theorem, also known as the Nyquist-Shannon Sampling Theorem [33, 45], is a fundamental concept in signal processing and digital communication that describes the conditions under which a continuous signal can be accurately represented or reconstructed from its discrete samples. To accurately reconstruct a continuous signal from its discrete samples without loss of information, the following conditions must be met:

Condition 1 *The continuous signal must be band-limited and may not contain any frequency components above a certain maximum frequency ν .*

Condition 2 *The sampling rate $\hat{\nu}$ must be at least twice the highest frequency present in the continuous signal: $\hat{\nu} \geq 2\nu$.*

In practice, to satisfy the constraints when reconstructing a signal from discrete samples, a low-pass or anti-aliasing filter is applied to the signal before sampling. The filter eliminates any frequency components above $\frac{\hat{\nu}}{2}$ and attenuates high-frequency content that could lead to aliasing.

3.2. 3D Gaussian Splatting

Prior works [18, 59] propose to represent a 3D scene as a set of scaled 3D Gaussian primitives $\{\mathcal{G}_k | k = 1, \dots, K\}$ and render an image using volume splatting. The geometry of each scaled 3D Gaussian $\mathcal{G}_k$ is parameterized by an opacity (scale) $\alpha_k \in [0, 1]$, center $\mathbf{p}_k \in \mathbb{R}^{3 \times 1}$ and covariance matrix $\Sigma_k \in \mathbb{R}^{3 \times 3}$ defined in world space:

$$\mathcal{G}_k(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\mathbf{p}_k)^T \Sigma_k^{-1}(\mathbf{x}-\mathbf{p}_k)} \quad (1)$$

To constrain Σ_k to the space of valid covariance matrices, a semi-definite parameterization $\Sigma_k = \mathbf{O}_k \mathbf{s}_k \mathbf{s}_k^T \mathbf{O}_k^T$ is used. Here, $\mathbf{s} \in \mathbb{R}^3$ is a scaling vector and $\mathbf{O} \in \mathbb{R}^{3 \times 3}$ is a rotation matrix, parameterized by a quaternion [18].

To render an image for a given view point defined by rotation $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ and translation $\mathbf{t} \in \mathbb{R}^3$, the 3D Gaussians $\{\mathcal{G}_k\}$ are first transformed into camera coordinates:

$$\mathbf{p}'_k = \mathbf{R} \mathbf{p}_k + \mathbf{t}, \quad \Sigma'_k = \mathbf{R} \Sigma_k \mathbf{R}^T \quad (2)$$

Afterwards, they are projected to ray space via a local affine transformation

$$\Sigma_k'' = \mathbf{J}_k \Sigma_k' \mathbf{J}_k^T \quad (3)$$

where the Jacobian matrix $\mathbf{J}_k$ is an affine approximation to the projective transformation defined by the center of the 3D Gaussian $\mathbf{p}_k'$. By skipping the third row and column of Σ_k'' , we obtain a 2D covariance matrix Σ_k^{2D} in ray space, and we use $\mathcal{G}_k^{2D}$ to refer to the corresponding scaled 2D Gaussian, see [18] for details.

Finally, 3DGS [18] utilizes spherical harmonics to model view-dependent color $\mathbf{c}_k$ and renders image via alpha blending according to the primitive's depth order $1, \dots, K$:

$$\mathbf{c}(\mathbf{x}) = \sum_{k=1}^K \mathbf{c}_k \alpha_k \mathcal{G}_k^{2D}(\mathbf{x}) \prod_{j=1}^{k-1} (1 - \alpha_j \mathcal{G}_j^{2D}(\mathbf{x})) \quad (4)$$

Dilation: To avoid degenerate cases where the projected 2D Gaussians are too small in screen space, i.e., smaller than a pixel, the projected 2D Gaussians are dilated as follows:

$$\mathcal{G}_k^{2D}(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\mathbf{p}_k)^T (\Sigma_k^{2D} + s \mathbf{I})^{-1} (\mathbf{x}-\mathbf{p}_k)} \quad (5)$$

where $\mathbf{I}$ is a 2D identity matrix and s is a scalar dilation hyperparameter. Note that this operator adjusts the scale of the 2D Gaussian while leaving its maximum unchanged. As this effect is similar to that of dilation operators in morphology, we called it a 2D screen space dilation operation¹.

Reconstruction: As the rendering process is fast and differentiable, the 3D Gaussian parameters can be efficiently optimized using a multi-view loss. During optimization, 3D Gaussians are adaptively added and deleted to better represent the scene. We refer the reader to [18] for details.

4. Sensitivity to Sampling Rate

In traditional forward splatting, the centers $\mathbf{p}_k$ and colors $\mathbf{c}_k$ of Gaussian primitives are predetermined, whereas the 3D Gaussian covariance Σ_k are chosen empirically [42, 59]. In contrast, 3DGS [18], optimizes all parameters jointly through an inverse rendering framework by backpropagating a multi-view photometric loss.

We observe that this optimization suffers from ambiguities as illustrated in Figure 1 which shows a simple example involving one object and an image sensor with 5 pixels. Consider the 3D object in (a), its approximation by a 3D Gaussian and its projection into screen space (blue pixel). Due to screen space dilation (Eq. 5) with a Gaussian kernel (size ≈ 1 pixel), the degenerate 3D Gaussian represented by a Dirac δ function in (b) leads to a similar image. This illustrates that the scale of the 3D Gaussian is not properly constrained. In practice, due to its implicit shrinkage bias,

¹The dilation operation is not mentioned in original paper.

3DGS indeed systematically underestimates the scale parameter of 3D Gaussians during optimization.

While this does not affect rendering at similar sampling rates (cf. Figure 1 (a) vs. (b)), it leads to erosion effects when zooming in or moving the camera closer. This is because the dilated 2D Gaussians become smaller in screen space. In this case, the rendered image exhibits high-frequency artifacts, rendering object structures thinner than they actually appear as illustrated in Figure 1 (d).

Conversely, screen space dilation also negatively affects rendering when decreasing the sampling rate as illustrated in Figure 1 (c) which shows a zoomed-out version of (a). In this case, dilation spreads radiance in a physically incorrect way across pixels. Note that in (c), the area covered by the projection of the 3D object is smaller than a pixel, yet the dilated Gaussian is not attenuated, accumulating more light than what physically reaches the pixel. This leads to increased brightness and dilation artifacts which strongly degrade the appearance of the bicycle wheels' spokes.

The aforementioned scale ambiguity becomes particularly problematic in representations involving millions of Gaussians. However, simply discarding screen space dilation results in optimization challenges for complex scenes, such as those present in the Mip-NeRF 360 dataset [2], where a large number of small Gaussian are created by the density control mechanism [18], exceeding GPU capacity. Moreover, even if a model can be successfully trained without dilation, decreasing the sampling rate results in aliasing effects due to the lack of anti-aliasing [59].

5. Mip Gaussian Splatting

To overcome these challenges, we make two modifications to the original 3DGS model. In particular, we introduce a 3D smoothing filter that limits the frequency of the 3D representation to below half the maximum sampling rate determined by the training images, eliminating high frequency artifacts when zooming in. Moreover, we demonstrate that replacing 2D screen space dilation with a 2D Mip filter which approximates the box filter inherent to the physical imaging process and effectively mitigates aliasing and dilation issues. In combination, Mip-Splatting enables alias-free renderings² across various sampling rates. We now discuss the 3D smoothing and the 2D Mip filters in detail.

5.1. 3D Smoothing Filter

3D radiance field reconstruction from multi-view observations is a well-known ill-posed problem as multiple distinctly different reconstructions can result in the same 2D projections [2, 55, 56]. Our key insight is that the highest frequency of a reconstructed 3D scene is limited by

²Note that we use alias to refer to multiple artifacts discussed in the paper, including dilation, erosion, oversmoothing, high-frequency artifacts and aliasing itself.

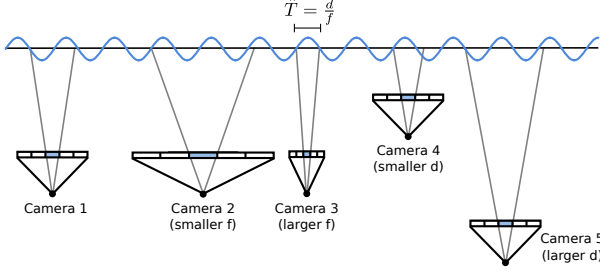


Figure 3. **Sampling limits.** A pixel corresponds to sampling interval $\hat{T}$. We band-limit the 3D Gaussians by the maximal sampling rate (i.e., minimal sampling interval) among all observations. This example shows 5 cameras at different depths d and with different focal lengths f . Here, camera 3 determines the minimal $\hat{T}$ and hence the maximal sampling rate $\hat{\nu}$.

the sampling rate defined by the training views. Following Nyquist’s theorem 3.1, we aim to constrain the maximum frequency of the 3D representation during optimization.

Multiview Frequency Bounds: Multi-view images are 2D projections of a continuous 3D scene. The discrete image grid determines where we sample points from the continuous 3D signal. This sampling rate is intrinsically related to the image resolution, camera focal length, and the scene’s distance from the camera. For an image with focal length f in pixel units, the sampling interval in screen space is 1. When this pixel interval is back-projected to the 3D world space, it results in a world space sampling interval $\hat{T}$ at a given depth d , with sampling frequency $\hat{\nu}$ as its inverse:

$$\hat{T} = \frac{1}{\hat{\nu}} = \frac{d}{f} \quad (6)$$

As posited by Nyquist’s theorem Section 3.1, given samples drawn at frequency $\hat{\nu}$, reconstruction algorithms are able to reconstruct components of the signal with frequencies up to $\frac{\hat{\nu}}{2}$, or $\frac{f}{2d}$. Consequently, a primitive smaller than $2\hat{T}$ may result in aliasing artifacts during the splatting process, since its size is below twice the sampling interval.

To simplify, we approximate depth d using the center of the primitive $\mathbf{p}_k$, and disregard the impact of occlusion for sampling interval estimation. Since the sampling rate of a primitive is depth-dependent and differs across cameras, we determine the maximal sampling rate for primitive k as

$$\hat{\nu}_k = \max \left(\left\{ \mathbb{1}_n(\mathbf{p}_k) \cdot \frac{f_n}{d_n} \right\}_{n=1}^N \right) \quad (7)$$

where N is the total number of images, $\mathbb{1}_n(\mathbf{p})$ is an indicator function that assesses the visibility of a primitive. It is true if the Gaussian center $\mathbf{p}_k$ falls within the view frustum of the n -th camera. Intuitively, we choose the sampling rate such that there exists at least one camera that is able to reconstruct the respective primitive. This process is illustrated in Figure 3 for $N = 5$. In our implementation,

we recompute the maximal sampling rate of each Gaussian primitive every m iterations as we found the 3D Gaussians centers remain relatively stable throughout the training.

3D Smoothing: Given the maximal sampling rate $\hat{\nu}_k$ for a primitive, we aim to constrain the maximal frequency of the 3D representation. This is achieved by applying a Gaussian low-pass filter $\mathcal{G}_{\text{low}}$ to each 3D Gaussian primitive $\mathcal{G}_k$ before projecting it onto screen space:

$$\mathcal{G}_k(\mathbf{x})_{\text{reg}} = (\mathcal{G}_k \otimes \mathcal{G}_{\text{low}})(\mathbf{x}) \quad (8)$$

This operation is efficient as convolving two Gaussians with covariance matrices Σ_1 and Σ_2 results in another Gaussian with variance $\Sigma_1 + \Sigma_2$. Hence,

$$\mathcal{G}_k(\mathbf{x})_{\text{reg}} = \sqrt{\frac{|\Sigma_k|}{|\Sigma_k + \frac{s}{\hat{\nu}_k} \cdot \mathbf{I}|}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{p}_k)^T (\Sigma_k + \frac{s}{\hat{\nu}_k} \cdot \mathbf{I})^{-1} (\mathbf{x} - \mathbf{p}_k)} \quad (9)$$

Here, s is a scalar hyperparameter to control the size of the filter. Note that the scale $\frac{s}{\hat{\nu}_k}$ of the 3D filters for each primitive are different as they depend on the training views in which they are visible. By employing 3D Gaussian smoothing, we ensure that the highest frequency component of any Gaussian does not exceed half of its maximal sampling rate for at least one camera. Note that $\mathcal{G}_{\text{low}}$ becomes an intrinsic part of the 3D representation, remaining constant post-training.

5.2. 2D Mip Filter

While our 3D smoothing filter effectively mitigates high-frequency artifacts [18, 59], rendering the reconstructed scene at lower sampling rates (e.g., zooming out or moving the camera further away) would still lead to aliasing. To overcome this, we replace the screen space dilation filter of 3DGS by a 2D Mip filter.

More specifically, we replicate the physical imaging process [29, 37, 48], where photons hitting a pixel on the camera sensor are integrated over the pixel’s area. While an ideal model would use a 2D box filter in image space, we approximate it with a 2D Gaussian filter for efficiency

$$\mathcal{G}_k^{2D}(\mathbf{x})_{\text{mip}} = \sqrt{\frac{|\Sigma_k^{2D}|}{|\Sigma_k^{2D} + s\mathbf{I}|}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{p}_k)^T (\Sigma_k^{2D} + s\mathbf{I})^{-1} (\mathbf{x} - \mathbf{p}_k)} \quad (10)$$

where s is chosen to cover a single pixel in screen space.

While our Mip filter shares similarities with the EWA filter [59], their underlying principles are distinct. Our Mip filter is designed to replicate the box filter in the imaging process, targeting an exact approximation of a single pixel. Conversely, the EWA filter’s role is to limit the frequency signal’s bandwidth, and the size of the filter is chosen empirically. The EWA paper [15, 59] even advocates for an identity covariance matrix, effectively occupying a 3x3 pixel region on the screen. However, this approach leads to overly

	PSNR $\uparrow$					SSIM $\uparrow$					LPIPS $\downarrow$				
	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.
NeRF w/o $\mathcal{L}_{\text{area}}$ [1, 28]	31.20	30.65	26.25	22.53	27.66	0.950	0.956	0.930	0.871	0.927	0.055	0.034	0.043	0.075	0.052
NeRF [28]	29.90	32.13	33.40	29.47	31.23	0.938	0.959	0.973	0.962	0.958	0.074	0.040	0.024	0.039	0.044
MipNeRF [1]	32.63	34.34	35.47	35.60	34.51	0.958	0.970	0.979	0.983	0.973	0.047	0.026	0.017	0.012	0.026
Plenoxels [11]	31.60	32.85	30.26	26.63	30.34	0.956	0.967	0.961	0.936	0.955	0.052	0.032	0.045	0.077	0.051
TensoRF [4]	32.11	33.03	30.45	26.80	30.60	0.956	0.966	0.962	0.939	0.956	0.056	0.038	0.047	0.076	0.054
Instant-NGP [32]	30.00	32.15	33.31	29.35	31.20	0.939	0.961	0.974	0.963	0.959	0.079	0.043	0.026	0.040	0.047
Tri-MipRF [17]*	32.65	34.24	35.02	35.53	34.36	0.958	0.971	0.980	0.987	0.974	0.047	0.027	0.018	0.012	0.026
3DGS [18]	28.79	30.66	31.64	27.98	29.77	0.943	0.962	0.972	0.960	0.960	0.065	0.038	0.025	0.031	0.040
3DGS [18] + EWA [59]	31.54	33.26	33.78	33.48	33.01	0.961	0.973	0.979	0.983	0.974	0.043	0.026	0.021	0.019	0.027
Mip-Splatting (ours)	32.81	34.49	35.45	35.50	34.56	0.967	0.977	0.983	0.988	0.979	0.035	0.019	0.013	0.010	0.019

Table 1. **Multi-scale Training and Multi-scale Testing on the Blender dataset [28].** Our approach achieves state-of-the-art performance in most metrics. It significantly outperforms 3DGS [18] and 3DGS + EWA [59]. * indicates that we retrain the model.

smooth results when zooming out as we will show in our experiments.

6. Experiments

We first present the implementation details of Mip-Splatting. We then assess its performance on the Blender dataset [28] and the challenging Mip-NeRF 360 dataset [2]. Finally, we discuss the limitations of our approach.

6.1. Implementation

We build our method upon the popular open-source 3DGS code base [18]³. Following [18], we train our models for 30K iterations across all scenes and use the same loss function, Gaussian density control strategy, schedule and hyperparameters. For efficiency, we recompute the sampling rate of each 3D Gaussian every $m = 100$ iterations. We choose the variance of our 2D Mip filter as 0.1, approximating a single pixel, and the variance of our 3D smoothing filter as 0.2, totaling 0.3 for a fair comparison with 3DGS [18] and 3DGS + EWA [59] which replaces the dilation of 3DGS with the EWA filter.

6.2. Evaluation on the Blender Dataset

Multi-scale Training and Multi-scale Testing: Following previous work [1, 17], we train our model with multi-scale data and evaluate on multi-scale data. Similar to [1, 17] where rays of full resolution images are sampled more frequently compared to lower resolution images, we sample 40 percent of full resolution images and 20 percent from other image resolutions each. Our quantitative evaluation is shown in Table 1. Our approach attains comparable or superior performance compared to state-of-the-art methods such as Mip-NeRF [1] and Tri-MipRF [17]. Notably, our method outperforms 3DGS [18] and 3DGS + EWA [59] by a substantial margin, owing to its 2D Mip filter.

Single-scale Training and Multi-scale Testing: Contrary to prior work that evaluates models trained on single-scale data at the same scale, we consider the an important new

setting that involves training on full-resolution images and rendering at various resolutions (i.e. $1\times$, $1/2$, $1/4$, and $1/8$) to mimic zoom-out effects. In the absence of a public benchmark for this setting, we trained all baseline methods ourselves. We use NeRFacc [23]’s implementation for NeRF [28], Instant-NGP [32], and TensoRF [4] for its efficiency. Official implementations were employed for Mip-NeRF [1], Tri-MipRF [17], and 3DGS [18]. The quantitative results, as presented in Table 2, indicate that our method significantly outperforms all existing state-of-the-art methods. A qualitative comparison is provided in Figure 4. Methods based on 3DGS [18] capture fine details more effectively than Mip-NeRF [1] and Tri-MipRF [17], but only at the original training scale. Notably, our method surpasses both 3DGS [18] and 3DGS + EWA [59] in rendering quality at lower resolutions. In particular, 3DGS [18] exhibits dilation artifacts. EWA splatting [59] uses a large low pass filter to limit the frequency of the rendered images, resulting in oversmoothed images, which becomes particularly apparent at lower resolutions.

6.3. Evaluation on the Mip-NeRF 360 Dataset

Single-scale Training and Multi-scale Testing: To simulate zoom-in effects, we train models on data downsampled by a factor of 8 and rendered at successively higher resolutions ($1\times$, $2\times$, $4\times$, and $8\times$). In the absence of a public benchmark for this setting, we trained all baseline methods ourselves. We use the official implementation for Mip-NeRF 360 [1] and 3DGS [18] and use a community reimplement for Zip-NeRF [3]⁴ as the code is not available. The results in Table 3 show that our method performs comparable to prior work at the training scale ($1\times$) and significantly exceeds all state-of-the-art methods at higher resolutions. As depicted in Figure 5, our method generates high fidelity imagery without high-frequency artifacts. Notably, both Mip-NeRF 360 [2] and Zip-NeRF [3] exhibit subpar performance at increased resolutions, likely due to their MLPs’ inability to extrapolate to out-of-distribution frequencies. While 3DGS [18] introduces notable erosion

³<https://github.com/graphdeco-inria/gaussian-splatting>

⁴<https://github.com/SuLvXiangXin/zipnerf-pytorch>

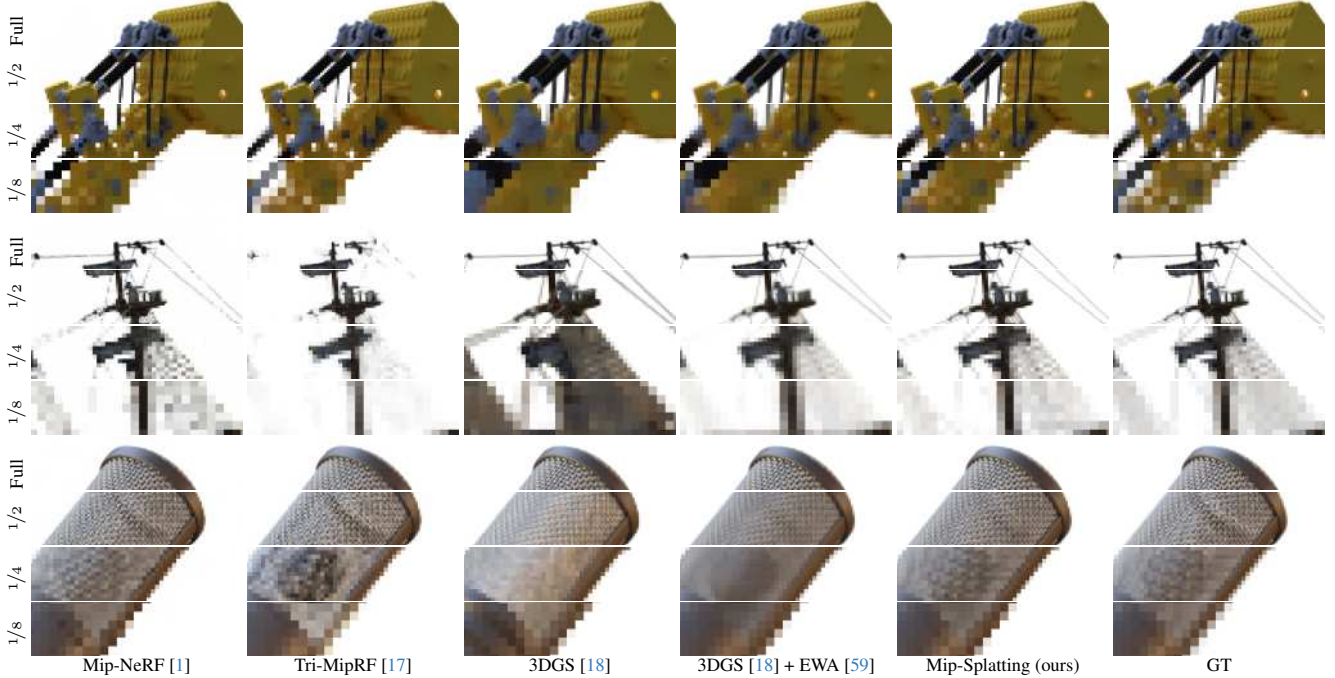


Figure 4. **Single-scale Training and Multi-scale Testing on the Blender Dataset [28].** All methods are trained at full resolution and evaluated at different (smaller) resolutions to mimic zoom-out. Methods based on 3DGS capture fine details better than Mip-NeRF [1] and Tri-MipRF [17] at training resolution. Mip-Splatting surpasses both 3DGS [18] and 3DGS + EWA [59] at lower resolutions.

	PSNR $\uparrow$					SSIM $\uparrow$					LPIPS $\downarrow$				
	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.
NeRF [28]	31.48	32.43	30.29	26.70	30.23	0.949	0.962	0.964	0.951	0.956	0.061	0.041	0.044	0.067	0.053
MipNeRF [1]	33.08	33.31	30.91	27.97	31.31	0.961	0.970	0.969	0.961	0.965	0.045	0.031	0.036	0.052	0.041
TensoRF [4]	32.53	32.91	30.01	26.45	30.48	0.960	0.969	0.965	0.948	0.961	0.044	0.031	0.044	0.073	0.048
Instant-NGP [32]	33.09	33.00	29.84	26.33	30.57	0.962	0.969	0.964	0.947	0.961	0.044	0.033	0.046	0.075	0.049
Tri-MipRF [17]	32.89	32.84	28.29	23.87	29.47	0.958	0.967	0.951	0.913	0.947	0.046	0.033	0.046	0.075	0.050
3DGS [18]	33.33	26.95	21.38	17.69	24.84	0.969	0.949	0.875	0.766	0.890	0.030	0.032	0.066	0.121	0.063
3DGS [18] + EWA [59]	33.51	31.66	27.82	24.63	29.40	0.969	0.971	0.959	0.940	0.960	0.032	0.024	0.033	0.047	0.034
Mip-Splatting (ours)	33.36	34.00	31.85	28.67	31.97	0.969	0.977	0.978	0.973	0.974	0.031	0.019	0.019	0.026	0.024

Table 2. **Single-scale Training and Multi-scale Testing on the Blender Dataset [28].** All methods are trained on full-resolution images and evaluated at four different (smaller) resolutions, with lower resolutions simulating zoom-out effects. While Mip-Splatting yields comparable results at training resolution, it significantly surpasses previous work at all other scales.

artifacts due to dilation operations, 3DGS + EWA [59] performs better while still yielding pronounced high-frequency artifacts. In contrast, our method avoids such artifacts, yielding aesthetically pleasing images that more closely resemble ground truth. It’s important to remark that rendering at higher resolutions is a super-resolution task, and models should not hallucinate high-frequency details absent from the training data.

Single-scale Training and Same-scale Testing: We further evaluate our method on the Mip-NeRF 360 dataset [2] following the widely used setting, where models are trained and tested at the same scale, with indoor scenes down-sampled by a factor of two and outdoor scenes by four. As shown in Table 4, our method performs on par with 3DGS [18] and 3DGS + EWA [59] in this challenging benchmark, without any decrease in performance. This con-

firms our method’s effectiveness to handle various settings.

6.4. Limitations

Our method employs a Gaussian filter as an approximation to a box filter for efficiency. However, this approximation introduces errors, particularly when the Gaussian is small in screen space. This issue correlates with our experimental findings, where increased zooming out leads to larger errors, as evidenced in Table 2. Additionally, there is a slight increase in training overhead as the sampling rate for each 3D Gaussian must be calculated every $m = 100$ iterations. Currently, this computation is performed using PyTorch [35] and a more efficient CUDA implementation could potentially reduce this overhead. Designing a better data structure for precomputing and storing the sampling rate, as it depends solely on the camera poses and intrinsic



Figure 5. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All models are trained on images downsampled by a factor of eight and rendered at full resolution to demonstrate zoom-in/moving closer effects. In contrast to prior work, Mip-Splatting renders images that closely approximate ground truth. Please also note the high-frequency artifacts of 3DGS + EWA [59].

	PSNR $\uparrow$					SSIM $\uparrow$					LPIPS $\downarrow$				
	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.
Instant-NGP [32]	26.79	24.76	24.27	24.27	25.02	0.746	0.639	0.626	0.698	0.677	0.239	0.367	0.445	0.475	0.382
mip-NeRF 360 [2]	29.26	25.18	24.16	24.10	25.67	0.860	0.727	0.670	0.706	0.741	0.122	0.260	0.370	0.428	0.295
zip-NeRF [3]	29.66	23.27	20.87	20.27	23.52	0.875	0.696	0.565	0.559	0.674	0.097	0.257	0.421	0.494	0.318
3DGS [18]	29.19	23.50	20.71	19.59	23.25	0.880	0.740	0.619	0.619	0.715	0.107	0.243	0.394	0.476	0.305
3DGS [18] + EWA [59]	29.30	25.90	23.70	22.81	25.43	0.880	0.775	0.667	0.643	0.741	0.114	0.236	0.369	0.449	0.292
Mip-Splatting (ours)	29.39	27.39	26.47	26.22	27.37	0.884	0.808	0.754	0.765	0.803	0.108	0.205	0.305	0.392	0.252

Table 3. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All methods are trained on the smallest scale (1 $\times$) and evaluated across four scales (1 $\times$, 2 $\times$, 4 $\times$, and 8 $\times$), with evaluations at higher sampling rates simulating zoom-in effects. While our method yields comparable results at the training resolution, it significantly surpasses all previous work at all other scales.

	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
NeRF [9, 28]	23.85	0.605	0.451
mip-NeRF [1]	24.04	0.616	0.441
NeRF++ [56]	25.11	0.676	0.375
Plenoxels [11]	23.08	0.626	0.463
Instant NGP [32, 52]	25.68	0.705	0.302
mip-NeRF 360 [2, 30]	27.57	0.793	0.234
Zip-NeRF [3]	28.54	0.828	0.189
3DGS [18]	27.21	0.815	0.214
3DGS [18]*	27.70	0.826	0.202
3DGS [18] + EWA [59]	27.77	0.826	0.206
Mip-Splatting (ours)	27.79	0.827	0.203

Table 4. **Single-scale Training and Same-scale Testing on the Mip-NeRF 360 dataset [2].** In the standard in-distribution setting, our approach demonstrates performance on par with many established techniques. * indicates that we retrain the model.

sics, is an avenue for future work. As mentioned before, the sampling rate computation is the only prerequisite during training and the 3D smoothing filter can be fused with the Gaussian primitives per Eq. 9, thereby eliminating any

additional overhead during rendering.

7. Conclusion

We presented Mip-Splatting, a modification to 3D Gaussian Splatting, which introduces two novel filters, namely a 3D smoothing filter and a 2D Mip filter, to achieve alias-free rendering at arbitrary scales. Our 3D smoothing filter effectively limits the maximal frequency of Gaussian primitives to match the sampling constraints imposed by the training images, while the 2D Mip filter approximates the box filter to simulate the physical imaging process. Our experimental results demonstrate that Mip-Splatting is competitive with state-of-the-art methods in terms of performance when training and testing at the same scale / sampling rate. Importantly, it significantly outperforms state-of-the-art methods in out-of-distribution scenarios, when testing at sampling rates different from training, resulting in better generalization to out-of-distribution camera poses and zoom factors.

References

- [1] Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. *ICCV*, 2021. 2, 3, 6, 7, 8, 1, 4, 5
- [2] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022. 2, 4, 6, 7, 8, 1, 3, 9
- [3] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. *Proc. of the IEEE International Conf. on Computer Vision (ICCV)*, 2023. 2, 3, 6, 8, 7, 9
- [4] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. 2022. 1, 2, 6, 7, 4, 5
- [5] Zhiqin Chen and Hao Zhang. Learning implicit fields for generative shape modeling. 2019. 2
- [6] Zhang Chen, Zhong Li, Liangchen Song, Lele Chen, Jingyi Yu, Junsong Yuan, and Yi Xu. Neurf: A neural fields representation with adaptive radial basis functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4182–4194, 2023. 2
- [7] Robert L. Cook. Stochastic sampling in computer graphics. *ACM Trans. Graph.*, 5(1):51–72, 1986. 3
- [8] Franklin C. Crow. Summed-area tables for texture mapping. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*, page 207–212, New York, NY, USA, 1984. Association for Computing Machinery. 3
- [9] Boyang Deng, Jonathan T. Barron, and Pratul P. Srinivasan. JaxNeRF: an efficient JAX implementation of NeRF, 2020. 8, 6
- [10] Robert A Drebin, Loren Carpenter, and Pat Hanrahan. Volume rendering. *ACM Siggraph Computer Graphics*, 22(4): 65–74, 1988. 2
- [11] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *CVPR*, 2022. 1, 2, 6, 8, 4
- [12] Steven J Gortler, Radek Grzeszczuk, Richard Szeliski, and Michael F Cohen. The lumigraph. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 453–464. 2023. 2
- [13] Markus Gross and Hanspeter Pfister. *Point-based graphics*. Elsevier, 2011. 3
- [14] Jeffrey P Grossman and William J Dally. Point sample rendering. In *Rendering Techniques' 98: Proceedings of the Eurographics Workshop in Vienna, Austria, June 29–July 1, 1998* 9, pages 181–192. Springer, 1998. 3
- [15] Paul S Heckbert. Fundamentals of texture mapping and image warping. 1989. 3, 5
- [16] Peter Hedman, Pratul P Srinivasan, Ben Mildenhall, Jonathan T Barron, and Paul Debevec. Baking neural radiance fields for real-time view synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5875–5884, 2021. 2
- [17] Wenbo Hu, Yuling Wang, Lin Ma, Bangbang Yang, Lin Gao, Xiao Liu, and Yuewen Ma. Tri-miprf: Tri-mip representation for efficient anti-aliasing neural radiance fields. In *Proc. of the IEEE International Conf. on Computer Vision (ICCV)*, 2023. 2, 3, 6, 7, 1, 4, 5, 8
- [18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. 1, 2, 3, 4, 5, 6, 7, 8, 9
- [19] Jonas Kulhanek and Torsten Sattler. Tetra-nerf: Representing neural radiance fields using tetrahedra. *arXiv preprint arXiv:2304.09987*, 2023. 2
- [20] Christoph Lassner and Michael Zollhofer. Pulsar: Efficient sphere-based neural rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1440–1449, 2021. 3
- [21] Marc Levoy. Efficient ray tracing of volume data. *ACM Transactions on Graphics (TOG)*, 9(3):245–261, 1990. 2
- [22] Marc Levoy and Pat Hanrahan. Light field rendering. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 441–452. 2023. 2
- [23] Ruilong Li, Hang Gao, Matthew Tancik, and Angjoo Kanazawa. Nerfacc: Efficient sampling accelerates nerfs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 18537–18546, 2023. 6
- [24] Lingjie Liu, Jiatao Gu, Kyaw Zaw Lin, Tat-Seng Chua, and Christian Theobalt. Neural sparse voxel fields. *NeurIPS*, 2020. 1, 2
- [25] Nelson Max. Optical models for direct volume rendering. *IEEE Transactions on Visualization and Computer Graphics*, 1(2):99–108, 1995. 2
- [26] Nelson Max and Min Chen. Local and global illumination in the volume rendering integral. Technical report, Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States), 2005. 2
- [27] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3d reconstruction in function space. 2019. 2
- [28] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020. 1, 2, 6, 7, 8, 3, 4, 5
- [29] Ben Mildenhall, Peter Hedman, Ricardo Martin-Brualla, Pratul P. Srinivasan, and Jonathan T. Barron. NeRF in the dark: High dynamic range view synthesis from noisy raw images. *CVPR*, 2022. 2, 5
- [30] Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, Peter Hedman, Ricardo Martin-Brualla, and Jonathan T. Barron. MultiNeRF: A Code Release for Mip-NeRF 360, Ref-NeRF, and RawNeRF, 2022. 8, 6
- [31] Klaus Mueller, Torsten Moller, J Edward Swan, Roger Crawfis, Naeem Shareef, and Roni Yagel. Splatting errors and antialiasing. *IEEE Transactions on Visualization and Computer Graphics*, 4(2):178–191, 1998. 3
- [32] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):102:1–102:15, 2022. 1, 2, 6, 7, 8, 4, 5

- [33] Harry Nyquist. Certain topics in telegraph transmission theory. *Transactions of the American Institute of Electrical Engineers*, 1928. 2, 3
- [34] Jeong Joon Park, Peter Florence, Julian Straub, Richard A. Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. 2019. 2
- [35] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 7
- [36] Songyou Peng, Chiyu "Max" Jiang, Yiyi Liao, Michael Niemeyer, Marc Pollefeys, and Andreas Geiger. Shape as points: A differentiable poisson solver. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 3
- [37] Steve Hollasch Peter Shirley, Trevor David Black. Ray tracing in one weekend, 2023. 2, 5
- [38] Hanspeter Pfister, Matthias Zwicker, Jeroen Van Baar, and Markus Gross. Surfels: Surface elements as rendering primitives. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, pages 335–342, 2000. 3
- [39] Sergey Prokudin, Qianli Ma, Maxime Raafat, Julien Valentin, and Siyu Tang. Dynamic point fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7964–7976, 2023. 3
- [40] Christian Reiser, Songyou Peng, Yiyi Liao, and Andreas Geiger. Kilonerf: Speeding up neural radiance fields with thousands of tiny mlps. 2021. 2
- [41] Christian Reiser, Richard Szeliski, Dor Verbin, Pratul P. Srinivasan, Ben Mildenhall, Andreas Geiger, Jonathan T. Barron, and Peter Hedman. Merf: Memory-efficient radiance fields for real-time view synthesis in unbounded scenes. *SIGGRAPH*, 2023. 2
- [42] Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer Graphics Forum*, pages 461–470. Wiley Online Library, 2002. 4
- [43] Darius Rückert, Linus Franke, and Marc Stamminger. Adop: Approximate differentiable one-pixel point rendering. *arXiv preprint arXiv:2110.06635*, 2021. 3
- [44] Miguel Sainz and Renato Pajarola. Point-based rendering techniques. *Computers & Graphics*, 28(6):869–879, 2004. 3
- [45] Claude E Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 1949. 2, 3
- [46] Cheng Sun, Min Sun, and Hwann-Tzong Chen. Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction. In *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2022. 1, 2
- [47] J Edward Swan, Klaus Mueller, Torsten Moller, N Shareel, Roger Crawfis, and Roni Yagel. An anti-aliasing technique for splatting. In *Proceedings. Visualization'97 (Cat. No. 97CB36155)*, pages 197–204. IEEE, 1997. 3
- [48] Richard Szeliski. *Computer vision: algorithms and applications*. Springer Nature, 2022. 2, 5
- [49] Olivia Wiles, Georgia Gkioxari, Richard Szeliski, and Justin Johnson. SynSin: End-to-end view synthesis from a single image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3
- [50] Lance Williams. Pyramidal parametrics. page 1–11, New York, NY, USA, 1983. Association for Computing Machinery. 3
- [51] Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-nerf: Point-based neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5438–5448, 2022. 2
- [52] Lior Yariv, Peter Hedman, Christian Reiser, Dor Verbin, Pratul P. Srinivasan, Richard Szeliski, Jonathan T. Barron, and Ben Mildenhall. BakedSDF: Meshing neural SDFs for real-time view synthesis. *arXiv*, 2023. 2, 8, 6
- [53] Wang Yifan, Felice Serena, Shihao Wu, Cengiz Öztireli, and Olga Sorkine-Hornung. Differentiable surface splatting for point-based geometry processing. *ACM Transactions on Graphics (proceedings of ACM SIGGRAPH ASIA)*, 38(6), 2019. 3
- [54] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. Plenotrees for real-time rendering of neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5752–5761, 2021. 2
- [55] Zehao Yu, Songyou Peng, Michael Niemeyer, Torsten Sattler, and Andreas Geiger. Monosdf: Exploring monocular geometric cues for neural implicit surface reconstruction. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 4
- [56] Kai Zhang, Gernot Riegler, Noah Snavely, and Vladlen Koltun. Nerf++: Analyzing and improving neural radiance fields. *arXiv:2010.07492*, 2020. 4, 8, 6
- [57] Yufeng Zheng, Wang Yifan, Gordon Wetzstein, Michael J. Black, and Otmar Hilliges. Pointavatar: Deformable point-based head avatars from videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 3
- [58] Yiyu Zhuang, Qi Zhang, Ying Feng, Hao Zhu, Yao Yao, Xiaoyu Li, Yan-Pei Cao, Ying Shan, and Xun Cao. Anti-aliased neural implicit surfaces with encoding level of detail. *arXiv preprint arXiv:2309.10336*, 2023. 3
- [59] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pages 29–538. IEEE, 2001. 2, 3, 4, 5, 6, 7, 8, 1, 9
- [60] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Surface splatting. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pages 371–378, 2001. 3

Mip-Splatting: Alias-free 3D Gaussian Splatting

Supplementary Material

In this **supplementary document**, we first present ablation studies of Mip-Splatting in Section 8. Next, we report additional quantitative and quality results in Section 9.

8. Ablation

In this section, we evaluate the effectiveness of our 3D smoothing filter and 2D Mip filter in Section 8.1 and Section 8.2. Then, we present an additional experiment to evaluate both zoom-in and zoom-out effects in the same dataset in Section 8.3.

8.1. Effectiveness of the 3D Smoothing Filter

To evaluate the effectiveness of the 3D smoothing filter, we conduct an ablation with the single-scale training and multi-scale testing setting to simulate zoom-in effects in the Mip-NeRF 360 dataset [2]. The quantitative result is presented in Table 5. Omitting the 3D smoothing filter results in high-frequency artifacts when rendering higher resolution image, as depicted in Figure 6. Excluding the 2D Mip filter causes a slight decline in performance as this filter’s role is mainly for mitigating zoom-out artifacts, as we will shown next. The absence of both the 3D smoothing filter and the 2D Mip filter leads to an excessive generation of small Gaussian primitives, due to the density control mechanism, resulting in out of memory error even on an A100 GPU with 40GB memory. Hence, we don’t report the result.

8.2. Effectiveness of the 2D Mip Filter

To evaluate the effectiveness of the 2D Mip filter, we perform an ablation study with the single-scale training and multi-scale testing setting to simulate zoom-out effects in the Blender dataset [28]. The quantitative results are shown in Table 6. Upon removing the dilation operation from 3DGS [18] (*3DGS - Dilation*), the dilation effects are eliminated, outperforming 3DGS in this context. However, it also results in aliasing artifacts due to a lack of anti-aliasing. Mip-Splatting outperforms all baseline methods by a large margin. Removing the 2D Mip filter results in a notable decline in performance, validating its critical role in anti-aliasing. Without the 3D smoothing filter, it still produces alias-free rendering as the 3D filter aims at addressing the high-frequency artifacts when zooming in.

8.3. Single-scale Training and Multi-scale Testing

In the main paper, we evaluate the zoom-out effects by rendering lower resolution images on the Blender dataset [28] following [1, 17] and simulating the zoom-in effects by rendering higher resolution images on the Mip-NeRF 360

dataset [2]. Here we present an addition experiment evaluating both zoom-out and zoom-in effects on the Mip-NeRF 360 dataset [2]. We use the images downsampled by a factor of 4 for training and evaluate it at multiple resolutions ($1/4\times$, $1/2\times$, $1\times$, $2\times$, $4\times$). The quantitative results are presented in Table 7 and the qualitative comparison is shown in Figure 7. Mip-Splatting significantly outperforms 3DGS [18] and 3DGS + EWA [59] in rendering quality when zooming in and out, which is consistent to our main results. Further, removing our 3D smoothing filter leads to high-frequency artifacts, while removing our 2D Mip-filter results in aliasing artifacts, as evidenced in Figure 7.

9. Additional Results

In this section, we provide more qualitative and quantitative results on the Blender dataset [28] in Section 9.1 and the Mip-NeRF 360 dataset [2] in Section 9.2.

9.1. Blender Dataset

We evaluate Mip-Splatting under two different settings in the Blender dataset [28]. For *multi-scale training and multi-scale testing*, the quantitative results are compiled in Table 8, where Mip-Splatting achieves state-of-the-art performance. Additionally, per-scene metrics for *single-scale training and multi-scale testing* are presented in Table 9. A qualitative comparison against leading methods is shown in Figure 8. Mip-Splatting outperforms both 3DGS [18] and 3DGS + EWA [59], particularly noticeable when zooming out, i.e. at lower resolutions.

9.2. Mip-NeRF 360 Dataset

We further evaluate Mip-Splatting on the Mip-NeRF 360 dataset [2] across two experimental setups. In the first setup, we follow the standard approach where models are trained and evaluated at the same scale, with indoor scenes downsampled by a factor of two and outdoor scenes by four. Quantitative results with per-scene metrics are shown in Table 10, our method performs on par with 3DGS [18] and 3DGS + EWA [59] in this challenging benchmark, without any decrease in performance.

In the second setup, models are trained on data downsampled by a factor of 8 and rendered at successively higher resolutions ($1\times$, $2\times$, $4\times$, and $8\times$) to simulate zoom-in effects. The quantitative results with per-scene metrics can be found in Table 11. Qualitative comparison with state-of-the-art methods are provided in Figure 9. Mip-Splatting effectively eliminates high-frequency artifacts, yielding high quality renderings that more closely resemble ground truth.

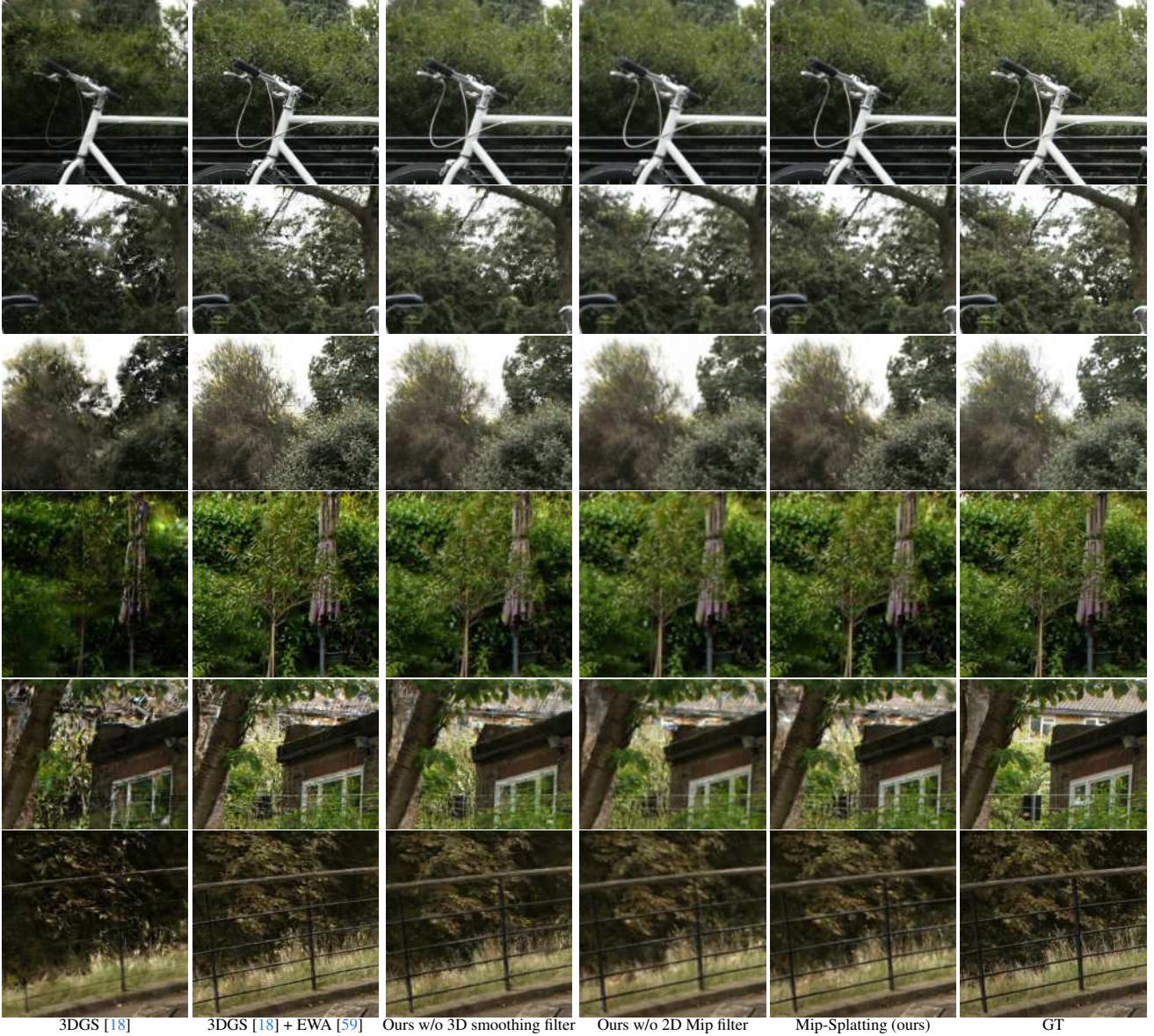


Figure 6. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All models are trained on images downsampled by a factor of 8 and rendered at full resolution to demonstrate zoom-in/moving closer effects. Removing the 3D smoothing filter results in high-frequency artifacts. Mip-Splatting renders images that closely approximate ground truth. Zoom in for a better view.

	PSNR $\uparrow$					SSIM $\uparrow$					LPIPS $\downarrow$				
	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.	1 $\times$ Res.	2 $\times$ Res.	4 $\times$ Res.	8 $\times$ Res.	Avg.
3DGS [18]	29.19	23.50	20.71	19.59	23.25	0.880	0.740	0.619	0.619	0.715	0.107	0.243	0.394	0.476	0.305
3DGS [18] + EWA [59]	29.30	25.90	23.70	22.81	25.43	0.880	0.775	0.667	0.643	0.741	0.114	0.236	0.369	0.449	0.292
Mip-Splatting (ours)	29.39	27.39	26.47	26.22	27.37	0.884	0.808	0.754	0.765	0.803	0.108	0.205	0.305	0.392	0.252
Mip-Splatting (ours) w/o 3D smoothing filter	29.41	27.09	25.83	25.38	26.93	0.881	0.795	0.722	0.713	0.778	0.107	0.214	0.342	0.424	0.272
Mip-Splatting (ours) w/o 2D Mip filter	29.29	27.22	26.31	26.08	27.23	0.882	0.798	0.742	0.759	0.795	0.107	0.214	0.319	0.407	0.262

Table 5. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All methods are trained on the smallest scale (1 $\times$) and evaluated across four scales (1 $\times$, 2 $\times$, 4 $\times$, and 8 $\times$), with evaluations at higher sampling rates simulating zoom-in effects. While our method yields comparable results at the training resolution, it significantly surpasses all previous work at all other scales. Omitting the 3D smoothing filter results in high-frequency artifacts when rendering higher resolution image as shown in 6, while the excluding the 2D Mip filter only causes a slight decline in performance as this filter’s role is mainly for mitigating zoom-out artifacts.

	PSNR $\uparrow$					SSIM $\uparrow$					LPIPS $\downarrow$				
	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.	Full Res.	1/2 Res.	1/4 Res.	1/8 Res.	Avg.
3DGS [18]	33.33	26.95	21.38	17.69	24.84	0.969	0.949	0.875	0.766	0.890	0.030	0.032	0.066	0.121	0.063
3DGS [18] + EWA [59]	33.51	31.66	27.82	24.63	29.40	0.969	0.971	0.959	0.940	0.960	0.032	0.024	0.033	0.047	0.034
3DGS [18] - Dilation	33.38	33.06	29.68	26.19	30.58	0.969	0.973	0.964	0.945	0.963	0.030	0.024	0.041	0.075	0.042
Mip-Splatting (ours)	33.36	34.00	31.85	28.67	31.97	0.969	0.977	0.978	0.973	0.974	0.031	0.019	0.019	0.026	0.024
Mip-Splatting (ours) w/o 3D smoothing filter	33.67	34.16	31.56	28.20	31.90	0.970	0.977	0.978	0.971	0.974	0.030	0.018	0.019	0.027	0.024
Mip-Splatting (ours) w/o 2D Mip filter	33.51	33.38	29.87	26.28	30.76	0.970	0.975	0.966	0.946	0.964	0.031	0.022	0.039	0.073	0.041

Table 6. **Single-scale Training and Multi-scale Testing on the Blender Dataset [28]**. All methods are trained on full-resolution images and evaluated at four different (smaller) resolutions, with lower resolutions simulating zoom-out effects. While Mip-Splatting yields comparable results at training resolution, it significantly surpasses previous work at all other scales. Removing the 2D Mip filter results in a notable decline in performance at lower resolutions, validating its critical role in anti-aliasing. Removing the 3D smoothing filter achieves similar performance since the 3D filter aims at addressing the high-frequency artifacts when zooming in.

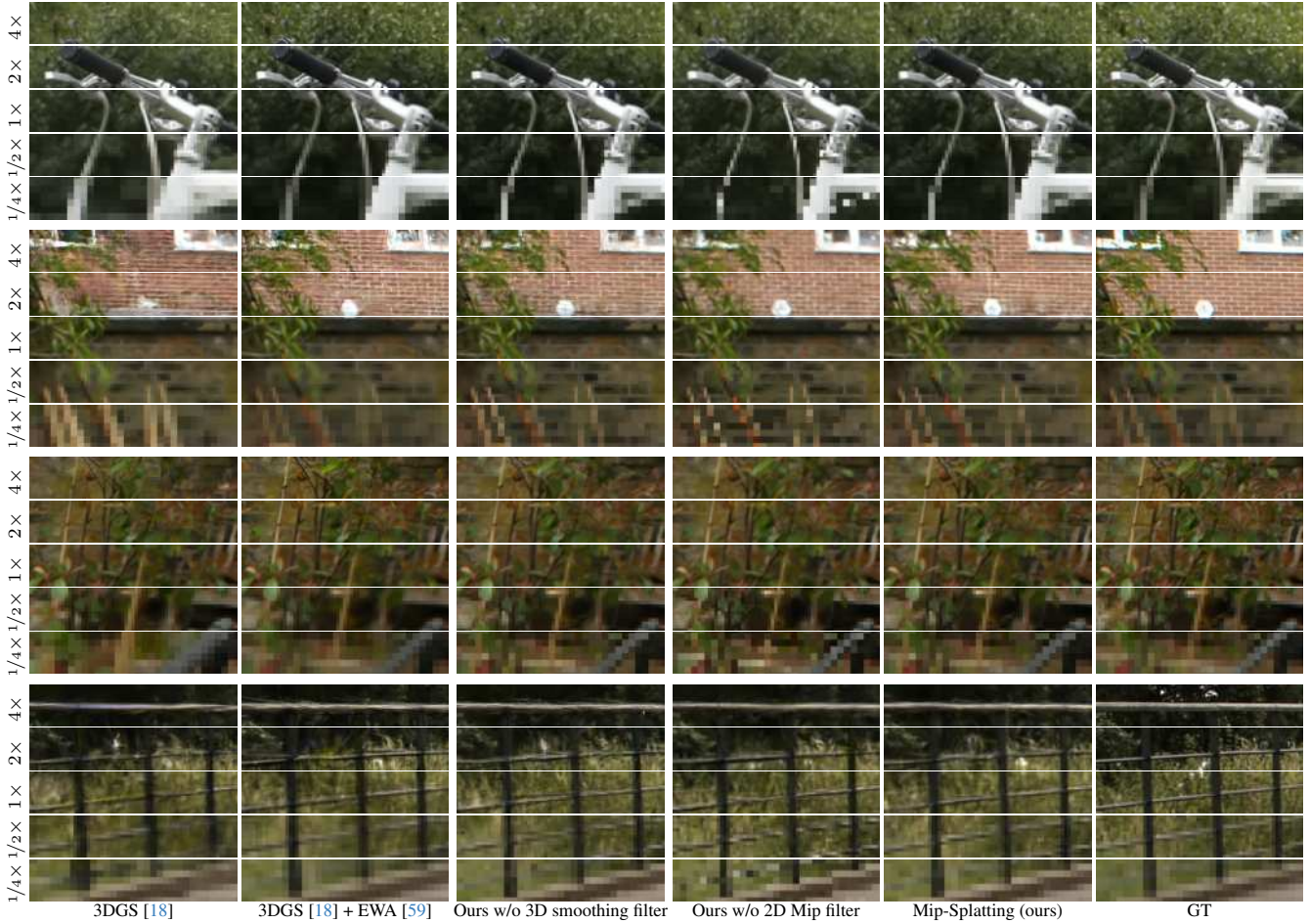


Figure 7. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2]**. All methods are trained at $1\times$ resolution and evaluated at different resolutions to mimic zoom-out ($1/4\times$ and $1/2\times$) and zoom-in ($2\times$ and $4\times$). Mip-Splatting surpasses both 3DGS [18] and 3DGS + EWA [59] across different resolutions. Removing 3D smoothing filter leads to high-frequency artifacts when zooming in, while omitting 2D Mip filter results in aliasing artifacts when zooming out.

	PSNR $\uparrow$						SSIM $\uparrow$						LPIPS $\downarrow$					
	$1/4$ Res.	$1/2$ Res.	$1\times$ Res.	$2\times$ Res.	$4\times$ Res.	Avg.	$1/4$ Res.	$1/2$ Res.	$1\times$ Res.	$2\times$ Res.	$4\times$ Res.	Avg.	$1/4$ Res.	$1/2$ Res.	$1\times$ Res.	$2\times$ Res.	$4\times$ Res.	Avg.
3DGS [18]	20.85	24.66	28.01	25.08	23.37	24.39	0.681	0.812	0.834	0.766	0.735	0.765	0.203	0.158	0.166	0.275	0.383	0.237
3DGS [18] + EWA [59]	27.40	28.39	28.09	26.43	25.30	27.12	0.888	0.871	0.833	0.774	0.738	0.821	0.103	0.126	0.171	0.276	0.385	0.212
Mip-Splatting (ours)	28.98	29.02	28.09	27.25	26.95	28.06	0.908	0.880	0.835	0.798	0.800	0.844	0.086	0.114	0.168	0.248	0.331	0.189
Mip-Splatting (ours) w/o 3D smoothing filter	28.69	28.94	28.05	27.06	26.61	27.87	0.905	0.879	0.833	0.790	0.780	0.837	0.088	0.115	0.168	0.261	0.359	0.198
Mip-Splatting (ours) w/o 2D Mip filter	26.09	28.04	28.05	27.27	27.00	27.29	0.815	0.856	0.834	0.798	0.802	0.821	0.167	0.132	0.167	0.249	0.335	0.210

Table 7. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All methods are trained on the middle scale ($1\times$) and evaluated across four scales ($1/4\times$, $1/2\times$, $1\times$, $2\times$, and $4\times$), with evaluations at higher sampling rates simulating zoom-in effects. While our method yields comparable results at the training resolution, it significantly surpasses all previous work at all other scales. Omitting the 3D smoothing filter results in high-frequency artifacts when rendering higher resolution images, while removing the 2D Mip filter results in aliasing artifacts when rendering lower resolution images, as shown in Figure 7.

	PSNR								
	<i>chair</i>	<i>drums</i>	<i>ficus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF w/o $\mathcal{L}_{\text{area}}$ [1, 28]	29.92	23.27	27.15	32.00	27.75	26.30	28.40	26.46	27.66
NeRF [28]	33.39	25.87	30.37	35.64	31.65	30.18	32.60	30.09	31.23
MipNeRF [1]	37.14	27.02	33.19	39.31	35.74	32.56	38.04	33.08	34.51
Plenoxels [11]	32.79	25.25	30.28	34.65	31.26	28.33	31.53	28.59	30.34
TensoRF [4]	32.47	25.37	31.16	34.96	31.73	28.53	31.48	29.08	30.60
Instant-ngp [32]	32.95	26.43	30.41	35.87	31.83	29.31	32.58	30.23	31.20
Tri-MipRF [17]*	37.67	27.35	33.57	38.78	35.72	31.42	37.63	32.74	34.36
3DGS [18]	32.73	25.30	29.00	35.03	29.44	27.13	31.17	28.33	29.77
3DGS [18] + EWA [59]	35.77	27.14	33.65	37.74	32.75	30.21	35.21	31.63	33.01
Mip-Splatting (ours)	37.48	27.74	34.71	39.15	35.07	31.88	37.68	32.80	34.56

	SSIM								
	<i>chair</i>	<i>drums</i>	<i>ficus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF w/o $\mathcal{L}_{\text{area}}$ [1, 28]	0.944	0.891	0.942	0.959	0.926	0.934	0.958	0.861	0.927
NeRF [28]	0.971	0.932	0.971	0.979	0.965	0.967	0.980	0.900	0.958
MipNeRF [1]	0.988	0.945	0.984	0.988	0.984	0.977	0.993	0.922	0.973
Plenoxels [11]	0.968	0.929	0.972	0.976	0.964	0.959	0.979	0.892	0.955
TensoRF [4]	0.967	0.930	0.974	0.977	0.967	0.957	0.978	0.895	0.956
Instant-ngp [32]	0.971	0.940	0.973	0.979	0.966	0.959	0.981	0.904	0.959
Tri-MipRF [17]*	0.990	0.951	0.985	0.988	0.986	0.969	0.992	0.929	0.974
3DGS [18]	0.976	0.941	0.968	0.982	0.964	0.956	0.979	0.910	0.960
3DGS [18] + EWA [59]	0.986	0.958	0.988	0.988	0.979	0.972	0.990	0.929	0.974
Mip-Splatting (ours)	0.991	0.963	0.990	0.990	0.987	0.978	0.994	0.936	0.979

	LPIPS								
	<i>chair</i>	<i>drums</i>	<i>ficus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF w/o $\mathcal{L}_{\text{area}}$ [1, 28]	0.035	0.069	0.032	0.028	0.041	0.045	0.031	0.095	0.052
NeRF [28]	0.028	0.059	0.026	0.024	0.035	0.033	0.025	0.085	0.044
MipNeRF [1]	0.011	0.044	0.014	0.012	0.013	0.019	0.007	0.062	0.026
Plenoxels [11]	0.040	0.070	0.032	0.037	0.038	0.055	0.036	0.104	0.051
TensoRF [4]	0.042	0.075	0.032	0.035	0.036	0.063	0.040	0.112	0.054
Instant-ngp [32]	0.035	0.066	0.029	0.028	0.040	0.051	0.032	0.095	0.047
Tri-MipRF [17]*	0.011	0.046	0.016	0.014	0.013	0.033	0.008	0.069	0.026
3DGS [18]	0.025	0.056	0.030	0.022	0.038	0.040	0.023	0.086	0.040
3DGS [18] + EWA [59]	0.017	0.039	0.013	0.016	0.024	0.026	0.011	0.070	0.027
Mip-Splatting (ours)	0.010	0.031	0.009	0.011	0.012	0.018	0.005	0.059	0.019

Table 8. **Multi-scale Training and Multi-scale Testing on the the Blender dataset [28].** For each scene, we report the arithmetic mean of each metric averaged over the 4 scales used in the dataset.

	PSNR								
	<i>chair</i>	<i>drums</i>	<i>figus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF [28]	31.99	25.31	30.74	34.45	30.69	28.86	31.41	28.36	30.23
MipNeRF [1]	32.89	25.58	31.80	35.40	32.24	29.46	33.26	29.88	31.31
TensoRF [4]	32.17	25.51	31.19	34.69	31.46	28.60	31.50	28.71	30.48
Instant-ngp [32]	32.18	25.05	31.32	34.85	31.53	28.59	32.15	28.84	30.57
Tri-MipRF [17]	32.48	24.01	28.41	34.45	30.41	27.82	31.19	27.02	29.47
3DGS [18]	26.81	21.17	26.02	28.80	25.36	23.10	24.39	23.05	24.84
3DGS [18] + EWA [59]	32.85	24.91	31.94	33.33	29.76	27.36	27.68	27.41	29.40
Mip-Splatting (ours)	35.69	26.50	32.99	36.18	32.76	30.01	31.66	29.98	31.97

	SSIM								
	<i>chair</i>	<i>drums</i>	<i>figus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF [28]	0.968	0.936	0.976	0.977	0.963	0.964	0.980	0.887	0.956
MipNeRF [1]	0.974	0.939	0.981	0.982	0.973	0.969	0.987	0.915	0.965
TensoRF [4]	0.970	0.938	0.978	0.979	0.970	0.963	0.981	0.906	0.961
Instant-ngp [32]	0.970	0.935	0.977	0.980	0.969	0.962	0.982	0.909	0.961
Tri-MipRF [17]	0.971	0.908	0.957	0.975	0.957	0.953	0.975	0.883	0.947
3DGS [18]	0.915	0.851	0.921	0.930	0.882	0.882	0.909	0.827	0.890
3DGS [18] + EWA [59]	0.978	0.942	0.983	0.977	0.964	0.958	0.963	0.912	0.960
Mip-Splatting (ours)	0.988	0.958	0.988	0.987	0.982	0.974	0.986	0.930	0.974

	LPIPS								
	<i>chair</i>	<i>drums</i>	<i>figus</i>	<i>hotdog</i>	<i>lego</i>	<i>materials</i>	<i>mic</i>	<i>ship</i>	<i>Average</i>
NeRF [28]	0.040	0.067	0.027	0.034	0.043	0.049	0.035	0.132	0.053
MipNeRF [1]	0.033	0.062	0.022	0.025	0.030	0.041	0.023	0.092	0.041
TensoRF [4]	0.036	0.066	0.027	0.030	0.035	0.052	0.034	0.102	0.048
Instant-ngp [32]	0.036	0.074	0.035	0.030	0.035	0.054	0.034	0.096	0.049
Tri-MipRF [17]	0.026	0.086	0.041	0.023	0.036	0.048	0.023	0.117	0.050
3DGS [18]	0.047	0.087	0.055	0.034	0.064	0.055	0.046	0.113	0.063
3DGS [18] + EWA [59]	0.023	0.051	0.017	0.018	0.033	0.027	0.024	0.077	0.034
Mip-Splatting (ours)	0.014	0.035	0.012	0.014	0.016	0.019	0.015	0.066	0.024

Table 9. **Single-scale Training and Multi-scale Testing on the the Blender dataset [28]**. For each scene, we report the arithmetic mean of each metric averaged over the four scales used in the dataset.

	PSNR									
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>	
NeRF [9, 28]	21.76	19.40	23.11	21.73	21.28	28.56	25.67	26.31	26.81	
mip-NeRF [1]	21.69	19.31	23.16	23.10	21.21	28.73	25.59	26.47	27.13	
NeRF++ [56]	22.64	20.31	24.32	24.34	22.20	28.87	26.38	27.80	29.15	
Plenoxels [11]	21.91	20.10	23.49	20.661	22.25	27.59	23.62	23.42	24.67	
Instant NGP [32, 52]	22.79	19.19	25.26	24.80	22.46	30.31	26.21	29.00	31.08	
mip-NeRF 360 [2, 30]	24.40	21.64	26.94	26.36	22.81	31.40	29.44	32.02	33.11	
Zip-NeRF [3]	25.80	22.40	28.20	27.55	23.89	32.65	29.38	32.50	34.46	
3DGS [18]	25.25	21.52	27.41	26.55	22.49	30.63	28.70	30.32	31.98	
3DGS [18]*	25.63	21.77	27.70	26.87	22.75	31.69	29.08	31.56	32.29	
3DGS [18] + EWA [59]	25.64	21.86	27.65	26.87	22.91	31.68	29.21	31.59	32.51	
Mip-Splatting (ours)	25.72	21.93	27.76	26.94	22.98	31.74	29.16	31.55	32.31	

	SSIM									
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>	
NeRF [9, 28]	0.455	0.376	0.546	0.453	0.459	0.843	0.775	0.749	0.792	
mip-NeRF [1]	0.454	0.373	0.543	0.517	0.466	0.851	0.779	0.745	0.818	
NeRF++ [56]	0.526	0.453	0.635	0.594	0.530	0.852	0.802	0.816	0.876	
Plenoxels [11]	0.496	0.431	0.606	0.523	0.509	0.842	0.759	0.648	0.814	
Instant NGP [32, 52]	0.540	0.378	0.709	0.654	0.547	0.893	0.845	0.857	0.924	
mip-NeRF 360 [2, 30]	0.693	0.583	0.816	0.746	0.632	0.913	0.895	0.920	0.939	
Zip-NeRF [3]	0.769	0.642	0.860	0.800	0.681	0.925	0.902	0.928	0.949	
3DGS [18]	0.771	0.605	0.868	0.775	0.638	0.914	0.905	0.922	0.938	
3DGS [18]*	0.777	0.622	0.873	0.783	0.652	0.928	0.916	0.933	0.948	
3DGS [18] + EWA [59]	0.777	0.620	0.871	0.784	0.655	0.927	0.916	0.933	0.948	
Mip-Splatting (ours)	0.780	0.623	0.875	0.786	0.655	0.928	0.916	0.933	0.948	

	LPIPS									
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>	
NeRF [9, 28]	0.536	0.529	0.415	0.551	0.546	0.353	0.394	0.335	0.398	
mip-NeRF [1]	0.541	0.535	0.422	0.490	0.538	0.346	0.390	0.336	0.370	
NeRF++ [56]	0.455	0.466	0.331	0.416	0.466	0.335	0.351	0.260	0.291	
Plenoxels [11]	0.506	0.521	0.3864	0.503	0.540	0.419	0.441	0.447	0.398	
Instant NGP [32, 52]	0.398	0.441	0.255	0.339	0.420	0.242	0.255	0.170	0.198	
mip-NeRF 360 [2, 30]	0.289	0.345	0.164	0.254	0.338	0.211	0.203	0.126	0.177	
Zip-NeRF [3]	0.208	0.273	0.118	0.193	0.242	0.196	0.185	0.116	0.173	
3DGS [18]	0.205	0.336	0.103	0.210	0.317	0.220	0.204	0.129	0.205	
3DGS [18]*	0.205	0.329	0.103	0.208	0.318	0.192	0.178	0.113	0.174	
3DGS [18] + EWA [59]	0.213	0.335	0.111	0.210	0.325	0.192	0.179	0.113	0.173	
Mip-Splatting (ours)	0.206	0.331	0.103	0.209	0.320	0.192	0.179	0.113	0.173	

Table 10. **Single-scale Training and Single-scale Testing on the Mip-NeRF 360 dataset [2]**. Indoor scenes are downsampled by a factor of 2 and outdoor scenes by 4.

	PSNR								
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>
Instant-NGP [32]	22.51	20.25	24.65	23.15	22.24	29.48	26.18	27.10	29.66
mip-NeRF 360 [2]	24.21	21.60	25.82	25.59	22.78	22.95	27.72	28.78	31.63
zip-NeRF [3]	23.05	20.05	18.07	23.94	22.53	20.51	26.08	27.37	30.05
3DGS [18]	21.34	19.43	21.94	22.63	20.91	28.10	25.33	23.68	25.89
3DGS [18] + EWA [59]	23.74	20.94	24.69	24.81	21.93	29.80	27.23	27.07	28.63
Mip-Splatting (ours)	25.26	22.02	26.78	26.65	22.92	31.56	28.87	30.73	31.49

	SSIM								
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>
Instant-NGP [32]	0.538	0.473	0.647	0.590	0.544	0.868	0.795	0.764	0.877
mip-NeRF 360 [2]	0.662	0.567	0.716	0.715	0.628	0.795	0.845	0.828	0.910
zip-NeRF [3]	0.640	0.521	0.548	0.661	0.590	0.655	0.784	0.800	0.865
3DGS [18]	0.638	0.536	0.675	0.662	0.591	0.878	0.826	0.789	0.838
3DGS [18] + EWA [59]	0.671	0.563	0.718	0.693	0.608	0.889	0.843	0.813	0.874
Mip-Splatting (ours)	0.738	0.613	0.786	0.776	0.659	0.921	0.897	0.903	0.933

	LPIPS								
	<i>bicycle</i>	<i>flowers</i>	<i>garden</i>	<i>stump</i>	<i>treehill</i>	<i>room</i>	<i>counter</i>	<i>kitchen</i>	<i>bonsai</i>
Instant-NGP [32]	0.500	0.486	0.372	0.469	0.511	0.270	0.310	0.286	0.229
mip-NeRF 360 [2]	0.358	0.400	0.296	0.333	0.391	0.256	0.228	0.210	0.182
zip-NeRF [3]	0.353	0.397	0.346	0.349	0.366	0.302	0.277	0.232	0.236
3DGS [18]	0.336	0.406	0.295	0.353	0.406	0.223	0.239	0.245	0.242
3DGS [18] + EWA [59]	0.322	0.395	0.281	0.334	0.405	0.217	0.231	0.216	0.227
Mip-Splatting (ours)	0.281	0.373	0.233	0.281	0.369	0.193	0.199	0.165	0.176

Table 11. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 dataset [2].** All models are trained on images down-sampled by a factor of 8 and rendered at higher resolutions to simulate zoom-in effects.



Figure 8. **Single-scale Training and Multi-scale Testing on the Blender Dataset [28].** All methods are trained at full resolution and evaluated at different (smaller) resolutions to mimic zoom-out. Methods based on 3DGS capture fine details better than Mip-NeRF [1] and Tri-MipRF [17] at training resolution. Mip-Splatting surpasses both 3DGS [18] and 3DGS + EWA [59] at lower resolutions.

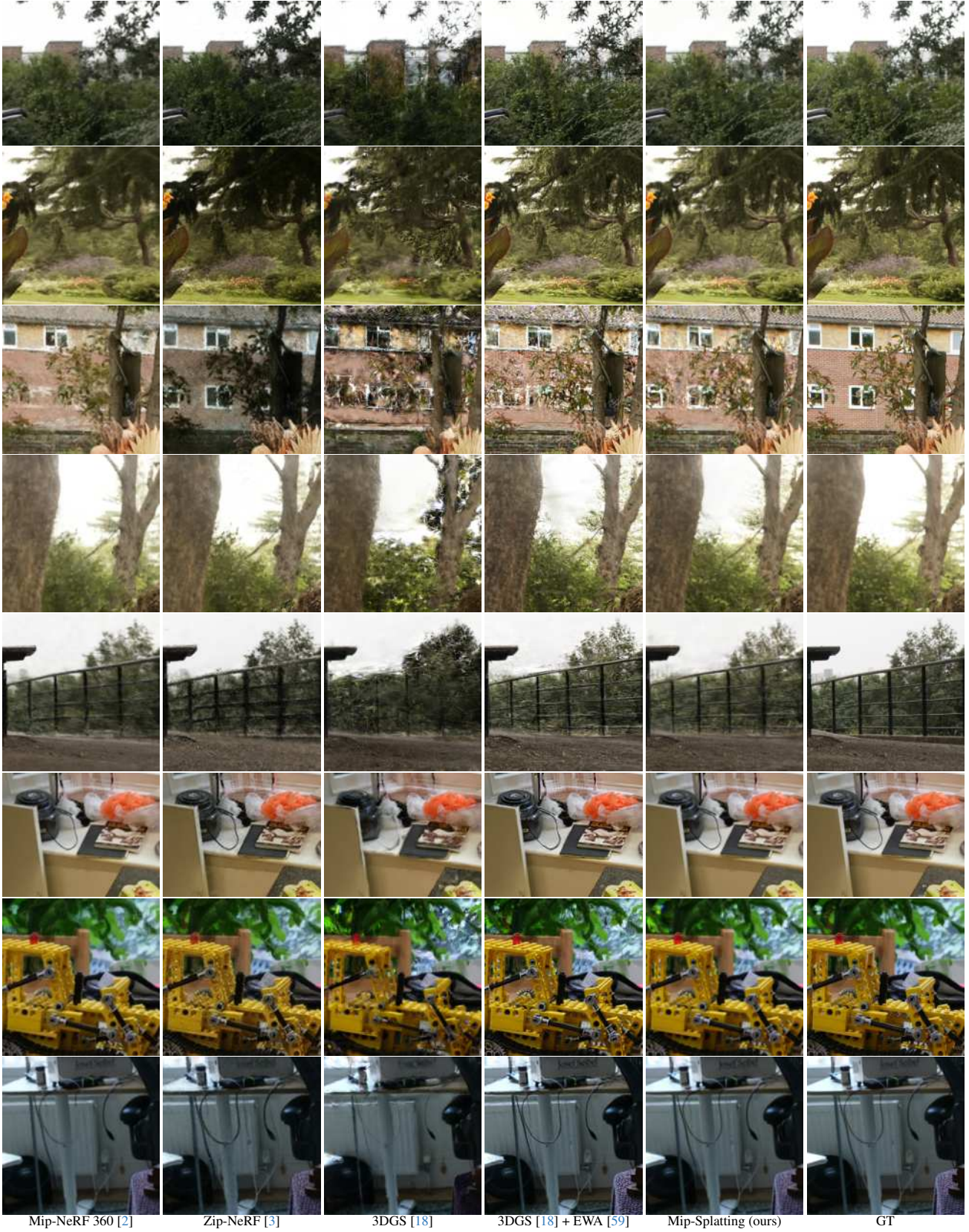


Figure 9. **Single-scale Training and Multi-scale Testing on the Mip-NeRF 360 Dataset [2].** All models are trained on images downsampled by a factor of eight and rendered at full resolution to demonstrate zoom-in/moving closer effects. In contrast to prior work, Mip-Splatting renders images that closely approximate ground truth. Please also note the high-frequency artifacts of 3DGS + EWA [59].